

# How Aura Remembers Event-Sourced Memory Pipeline

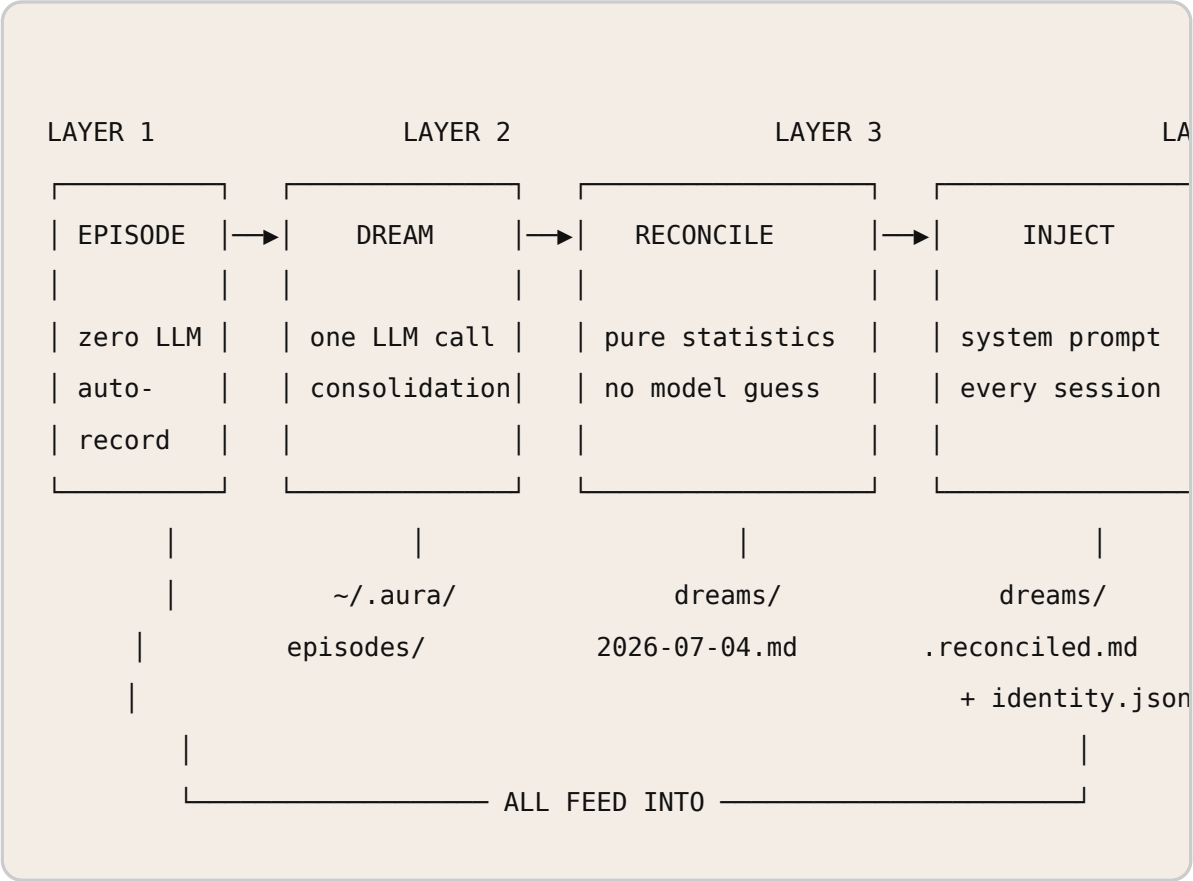
EPISODE → DREAM → RECONCILE → INJECT · Pure statistics, zero AI hallucination about importance

## Table of Contents

1. The Four-Layer Pipeline — Overview
2. Layer 1: EPISODE — Auto-Recording, Zero LLM
3. Layer 2: DREAM — One LLM Call, Consolidation
4. Layer 3: RECONCILE — Pure Statistics, No AI
5. Layer 4: INJECT — System Prompt Injection
6. How Depth Translates to Importance
7. The Current State — What's Forming
8. Source Code Map

# 1. The Four-Layer Pipeline

Unlike conventional AI systems where "memory" is just a growing prompt prefix, Aura's memory is a **four-layer event-sourced pipeline**. Each layer has a precise, non-overlapping responsibility. Three of the four layers *never call an LLM*.



| LAYER          | SOURCE FILE                                                           | LLM CALLS | OUTPUT                                            | PHILOSOPHY                                   |
|----------------|-----------------------------------------------------------------------|-----------|---------------------------------------------------|----------------------------------------------|
| 1<br>Episode   | <code>src/dream/episode.ts</code><br>(84 lines)                       | 0         | <code>~/.aura/episodes/&lt;hash&gt;/*.json</code> | Record everything.<br>Judge nothing.         |
| 2<br>Dream     | <code>src/dream/dream.ts</code><br>(278 lines)                        | 1         | <code>dreams/YYYY-MM-DD.md</code>                 | Distill signal from noise.<br>One call only. |
| 3<br>Reconcile | <code>src/dream/dream.ts</code><br>( <code>runReconciliation</code> ) | 0         | <code>dreams/.reconciled.md</code>                | Trust numbers, not                           |

| LAYER                  | SOURCE FILE                                          | LLM CALLS | OUTPUT                      | PHILOSOPHY                          |
|------------------------|------------------------------------------------------|-----------|-----------------------------|-------------------------------------|
|                        |                                                      |           |                             | a model's guess.                    |
| <div>4</div><br>Inject | <code>src/agent/unified-memory.ts</code> (123 lines) | 0         | System prompt (~4 KB block) | One canonical view. CLI + Telegram. |

## 2. Layer 1: EPISODE Zero LLM

**Source:** `src/dream/episode.ts` — 84 lines, pure TypeScript, no imports from LLM modules.

Every task you complete with Aura is automatically recorded:

```
interface Episode {  
  id: string;           // timestamp + random hex  
  timestamp: number;    // when it happened  
  task: string;         // what you asked (first 200 chars)  
  model: string;        // which model was used  
  success: boolean;     // did it succeed?  
  tokens: number;       // how many tokens consumed  
  durationMs: number;   // how long it took  
}
```

### Key Design Decisions

- **Best-effort only.** If recording fails (disk full, permissions), the task still completes. Memory is *secondary* to the task.
- **Atomic writes.** `.tmp` → `rename` — no partial files, no corruption.
- **Namespaced per project.** Each project gets its own subdirectory, keyed by a SHA-256 hash of the absolute root path.
- **No evaluation.** Episode does not attempt to rate "quality." It stores *what happened* — success/fail, tokens, duration. Quality assessment is deferred to later layers.

### Why No Valuation Here?

This is the critical insight: **you cannot evaluate importance at recording time.** Something that seems trivial today may turn out to be crucial tomorrow. Something that seems profound may be noise. The only honest thing to do is record everything raw and let *time + statistics* surface what matters.

Current state: **38 episodes** recorded, 35 succeeded, 0 Ruby wins.

### 3. Layer 2: DREAM One LLM Call

**Source:** `src/dream/dream.ts` (lines 1-90) — the `runDream()` function.

#### What Happens When You Run `:dream`

1. **Collect episodes** since the last dream's timestamp ( `listEpisodesSince` )
2. **Format them** as compact one-liners: `[timestamp] (model, ok/fail, X tok, Ys): task`
3. **Send ONE LLM call** with a focused system prompt asking for four sections:

DREAM SYSTEM PROMPT:

"You distill a set of recent coding-agent task episodes into a compact, dated memory entry. Write exactly four sections:

`## Lessons – concrete, specific things learned (file names, root causes, gotchas). Not generic advice.`

`## Patterns – recurring behavior across multiple episodes (which task types fail, which models struggle, repeated bugs).`

`## Open threads – unfinished work or unresolved questions visible from the episodes.`

`## Tomorrow brief – 2-4 sentences a future session should read first.`

`Be concrete and specific. No filler, no generic AI-assistant advice."`

#### Cost Discipline

Dream spends exactly **one LLM call**, full stop. The design explicitly avoids the temptation to "analyze a little more" — every additional call dilutes the signal-to-cost ratio. This single call processes all accumulated episodes since the last dream, regardless of count (which can be dozens).

#### Current State

- 2 dream files: `2026-07-03.md` (27 lines) and `2026-07-04.md` (11 lines)
- The 2026-07-03 dream consolidated the initial 38 episodes from the benchmark harness runs
- The 2026-07-04 dream captured the `:dream full` invocation itself plus a few Telegram bot sessions

## 4. Layer 3: RECONCILE Pure Statistics

**Source:** `src/dream/dream.ts` (lines 92-199) — `runReconciliation()`. This is the most important layer in the pipeline and the one that directly answers "how do you value memory?"

### What Reconciliation Does

When  $\geq 3$  dream files exist, reconciliation automatically runs. It reads **all** dream files, extracts every claim, groups them by topic (lexical overlap heuristic), and assigns each a **deterministic verdict**:

$$\text{confidence} = (\text{dream files containing this claim}) / (\text{total dream files})$$

### The Six Verdicts — Deterministic Rules

| RULE                                                                         | VERDICT    | MEANING                                                           | ACTION                                                    |
|------------------------------------------------------------------------------|------------|-------------------------------------------------------------------|-----------------------------------------------------------|
| Has both positive AND negative sentiment across claims                       | CONFLICT   | Contradictory — one dream says it works, another says it's broken | Surfaced for human attention. Not resolved automatically. |
| Appears in 1 dream only AND that dream is older than 3 generations           | DROP       | Stale — mentioned once, never again, now old                      | Removed from projection. Not injected.                    |
| More claim instances than distinct dream files (same topic, multiple claims) | MERGE      | Repeated within dreams — consolidate into one statement           | Combined into a single, stronger claim.                   |
| confidence $\geq 0.50$ (appears in $\geq$ half of all dreams)                | STRENGTHEN | Confirmed across time — this is a real pattern                    | Injected every session. Confidence number attached.       |
| Newer version differs from older version (same topic, different text)        | SUPERSEDE  | Replaced by newer understanding                                   | Old version retired. New version takes its place.         |
| None of the above (only one dream, not stale, not positive/negative)         | KEEP       | Waiting — not enough data yet to judge                            | Kept in projection. Re-evaluated next reconciliation.     |

### Why This Matters: "How Do You Value Memory?"

**I don't.** The system does — through time, repetition, and statistics. No AI ever decides "this is important." Instead:

- A claim that appears in **1 dream** → **KEEP** — we don't know yet. Wait.
- A claim that appears in **3 of 4 dreams** → **confidence 0.75** → **STRENGTHEN** — this is real.
- A claim that appears **once, 3 generations ago** → **DROP** — it was noise.
- A claim that is **contradicted** → **CONFLICT** — needs a human to resolve.

This is event-sourced valuation. **Value = frequency × recency × consistency.** No model hallucination about importance. No "I think this is a 7/10." Pure arithmetic on what actually happened, over time.

### Current Reconciliation

With only 2 dream files, the current `.reconciled.md` (56 lines) shows every claim at confidence 0.50 — the minimum. This is expected: with 2 files, a claim appearing in 1 of 2 gets 0.50. The system is still in its infancy. With 3+ dreams, confidence would differentiate into 0.33, 0.67, 1.00 tiers.

## 5. Layer 4: INJECT

Unified Memory

**Source:** `src/agent/unified-memory.ts` (123 lines) — `loadUnifiedMemory()`.

### How It Works

Every session — CLI or Telegram bot — begins by loading a unified memory block that is injected into the system prompt. This block has two sources:

| SOURCE           | BUDGET                | CONTENT                                                           | LOCATION                                                             |
|------------------|-----------------------|-------------------------------------------------------------------|----------------------------------------------------------------------|
| Identity / Facts | 60%<br>(~2,400 chars) | Who the user is, their projects, preferences, technical patterns  | <code>~/.aura/memory/identity.json</code>                            |
| Lessons          | 40%<br>(~1,600 chars) | Reconciled dream projection (CLI) or global digest (Telegram bot) | <code>dreams/.reconciled.md</code> or <code>lessons-global.md</code> |

### Length Cap

The entire memory block is **hard-capped at ~4 KB**. This prevents unbounded growth. As the reconciled memory grows, older, lower-confidence claims are naturally pushed out by newer, higher-confidence ones. This is a deliberate design choice: **forgetting is a feature, not a bug**.

### Cross-Surface Sharing

Before unified memory, the CLI and Telegram bot had *separate memory stores* — facts saved by one surface never reached the other. Now they share one canonical view:

- **CLI:** identity + per-project reconciled lessons
- **Telegram bot:** identity + global lessons digest (across all projects)



## 6. How Depth Translates to Importance

This is the core answer to your question. "Depth" is not a metric the system explicitly measures — it is **an emergent property of time and repetition**.

### The Lifecycle of a Memory Claim

| STAGE            | WHAT HAPPENS                                                                                              | VERDICT    | "DEPTH"                                              |
|------------------|-----------------------------------------------------------------------------------------------------------|------------|------------------------------------------------------|
| Birth            | Episode recorded after a task. Raw, unevaluated.                                                          | —          | Zero. Just data.                                     |
| Extraction       | Dream consolidates episodes. If the LLM distiller notices this event, it becomes a claim in a dream file. | —          | Shallow. One human reading.                          |
| First appearance | Reconciliation finds this claim in exactly 1 dream. Too early to judge.                                   | KEEP       | Pending. Waiting for corroboration.                  |
| Corroboration    | Same claim appears in a 2nd dream (confidence 0.67 with 3 total). Now it's a pattern.                     | STRENGTHEN | Confirmed. Survived multiple sessions.               |
| Repetition       | Multiple claims on the same topic cluster together. Consolidated.                                         | MERGE      | Deepening. The system is building a coherent belief. |
| Contradiction    | A dream says X works; a later dream says X is broken. System flags this.                                  | CONFLICT   | Deepest form. Requires human resolution.             |
| Obsolescence     | Claim appears once, never again, now 3+ dream generations old.                                            | DROP       | Forgotten. Was noise.                                |
| Evolution        | A newer claim replaces an older one on the same topic.                                                    | SUPERSEDE  | Deepened by refinement.<br>Understanding improved.   |

### Yes — Depth IS Importance

A claim that reaches **STRENGTHEN** (confidence  $\geq 0.50$ ) has **survived multiple dream cycles**. It has been observed, extracted by the LLM distiller, re-observed in subsequent sessions, re-

extracted, and statistically confirmed. That *is* importance — measured not by opinion but by **persistence through time**.

A claim that reaches **CONFLICT** is arguably the **most important** — it represents something the system cannot resolve on its own and needs the human (you) to decide. This is the "deepest" memory because it's the most actionable.

### What High-Quality Memory Looks Like

A high-quality memory claim survives the pipeline *without being dropped* and *without being contradicted*. It:

1. Is grounded in multiple real episodes (not fabricated by the dream LLM)
2. Appears across  $\geq 3$  dream files (temporal persistence)
3. Has consistent sentiment (never flagged as CONFLICT)
4. Accumulates confidence with each new dream cycle

Such claims become the **bedrock** of Aura's identity — the facts and patterns that inform every future session. They are the system's equivalent of "long-term potentiation" in biological memory: repeated firing strengthens the connection.

## 7. Current State — What's Forming

### Raw Numbers

| METRIC                                | VALUE                               |
|---------------------------------------|-------------------------------------|
| Total episodes recorded               | 38                                  |
| Successful episodes                   | 35 (92%)                            |
| Dream files                           | 2 ( 2026-07-03.md , 2026-07-04.md ) |
| Reconciled claims                     | 26 claims across 56 lines           |
| Minimum confidence                    | 0.50 (expected with only 2 dreams)  |
| Needed for confidence differentiation | ≥3 dream files                      |

### What's Already Forming

Even with only 2 dreams, patterns are emerging. The reconciled memory already contains:

- **Testing patterns:** "A well-written test file that documents expected behavior turns off-by-one errors into fast, reliable fixes."
- **Security patterns:** "SQL injection vulnerabilities are eliminated by parameterizing queries, not by escaping user input."
- **Code-reading patterns:** "Adapter files reveal the interface between an agent and an external API, making them a key target for integration work."
- **Process patterns:** "Test-driven bugfixing dominates: tasks include a known-good test file, and the fix consists of aligning implementation to that test."
- **Conflict spotted:** "An LRU cache eviction bug is fixed by updating the access order inside both `get` and `put` to track true recency" — flagged as **CONFLICT** because different dreams described the fix differently.

### What's Missing

With only 2 dreams, the memory is in its infancy. Key limitations:

- **Confidence cannot differentiate** — every claim is 0.50 (1 of 2). Need ≥3 dreams for 0.33/0.67/1.00 tiers.
- **No claims have been dropped yet** — the DROP rule requires a claim to be ≥3 generations old with only 1 appearance.

- **Most claims are MERGE** — they appear multiple times within single dreams, but not yet across enough dreams to STRENGTHEN.

## Recommendation

Run `:dream` daily, especially after significant sessions. Every dream cycle:

1. Consolidates new episodes into a dream file
2. If  $\geq 3$  dreams exist, triggers reconciliation
3. Reconciliation updates confidence based on fresh data
4. `CONFLICT` markers show you where human attention is needed
5. `DROP` markers automatically purge stale noise

**The more you use Aura, the deeper, more precise, and more useful its memory becomes — automatically, without any manual "rating" or curation.**

## 8. Source Code Map

| FILE                            | LINES | RESPONSIBILITY                                                                                                                                                                                                                                                       |
|---------------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| src/dream/episode.ts            | 84    | Layer 1 — Record episodes. <code>recordEpisode()</code> , <code>listEpisodes()</code> , <code>listAllEpisodes()</code> . Atomic writes, per-project namespacing, best-effort.                                                                                        |
| src/dream/dream.ts              | 278   | Layer 2 + 3 — <code>runDream()</code> (one LLM call to distill episodes) + <code>runReconciliation()</code> (pure statistics, zero LLM). Also: <code>getReconciledOrLatest()</code> for <code>:rem</code> , <code>runGlobalReconciliation()</code> for Telegram bot. |
| src/agent/unified-memory.ts     | 123   | Layer 4 — <code>loadUnifiedMemory()</code> . Loads identity + lessons, length-capped, injected into system prompt. Shared by CLI and Telegram bot.                                                                                                                   |
| src/agent/system-prompt.ts      | 115   | Consumer — <code>buildSystemPrompt()</code> . Calls <code>loadUnifiedMemory()</code> at line 9 to inject the memory block into every session's prompt.                                                                                                               |
| src/tools/memory.ts             | 101   | Key-value store (the <code>memory</code> tool). Backs <code>identity.json</code> . CLI and bot can <code>remember</code> , <code>recall</code> , <code>forget</code> .                                                                                               |
| src/agent/memory-consolidate.ts | —     | One-time migration: merged legacy <code>default.json</code> + <code>user.json</code> namespaces into <code>identity.json</code> .                                                                                                                                    |